

การติดตั้ง PREEMPT_RT Kernel บน Compute Module 5 (CM5)

1. โครงสร้างฮาร์ดแวร์ Raspberry Pi CM5 ที่เกี่ยวข้อง

การคอมไพล์ Real-Time Kernel บน Compute Module 5 (CM5) มีข้อได้เปรียบสูงเนื่องจากสถาปัตยกรรมฮาร์ดแวร์ระดับอุตสาหกรรมที่ทรงประสิทธิภาพ:

หน่วยประมวลผลหลัก (CPU): Broadcom BCM2712 (Quad-core ARM Cortex-A76) ความเร็ว 2.4GHz
รองรับสถาปัตยกรรมแบบ 64-bit (aarch64) มีความเร็วเทียบเท่าคอมพิวเตอร์ระดับเดสก์ท็อป

หน่วยความจำ (RAM): รองรับสูงสุด 16GB LPDDRX ช่วยให้มีพื้นที่ Buffer และ Cache เพียงพอในการรันคำสั่งคอมไพล์แบบขนานเต็ม 4 คอร์ (make -j4)

ข้อจำกัดขอร์สโค้ด: การทำ Real-Time Kernel จำเป็นต้องใช้ Source Code เฉพาะของทางคลัง Raspberry Pi (ไม่ใช่โครงสร้างหลักของ kernel.org) เพื่อรักษาระบบไดรเวอร์ควบคุมฮาร์ดแวร์และพอร์ตอินเทอร์เฟซจำเพาะของ CM5 ให้ทำงานร่วมกันได้อย่างสมบูรณ์

2. ขั้นตอนการเตรียมระบบและการติดตั้ง Dependencies

ก่อนเริ่มกระบวนการจัดเตรียมซอร์สโค้ด จำเป็นต้องติดตั้งเครื่องมือและแพ็คเกจพื้นฐานที่จำเป็นสำหรับกระบวนการคอมไพล์ (Build Essentials):

Bash

```
sudo apt update
```

```
sudo apt install -y build-essential bison flex libssl-dev libelf-dev libncurses-dev bc git wget xz-utils
```

3. การจัดการซอร์สโค้ดและการกำหนดค่าผ่าน Menuconfig

เนื่องจาก Compute Module 5 (CM5) ต้องการไดรเวอร์ฮาร์ดแวร์ที่จำเพาะเจาะจง ทางผู้พัฒนา Raspberry Pi

จึงได้ทำการควรวม (Integrate) ซอร์สโค้ดและ Real-Time Patch ไว้ในคลัง GitHub หลักอย่างเป็นทางการ

ผู้พัฒนาไม่จำเป็นต้องแยกดาวน์โหลดไฟล์ Patch (.patch.xz) มาติดตั้งเอง

ซึ่งช่วยลดความเสี่ยงจากการเกิดข้อผิดพลาดประเภท Reversed patch

3.1 การดึงซอร์สโค้ดจากคลัง (Cloning Source Code)

ทำการดึงข้อมูลซอร์สโค้ดตามเวอร์ชันของ Kernel ปัจจุบัน (ตัวอย่างอ้างอิงเวอร์ชันย่อย 6.18.y):

Bash

```
mkdir ~/pi_rt_kernel && cd ~/pi_rt_kernel
```

```
git clone --depth=1 --branch rpi-6.18.y https://github.com/raspberrypi/linux.git
```

```
cd ~/pi_rt_kernel/linux
```

3.2 การสร้าง Configuration พื้นฐานและการปรับแต่งผ่านอินเทอร์เฟซระบบ

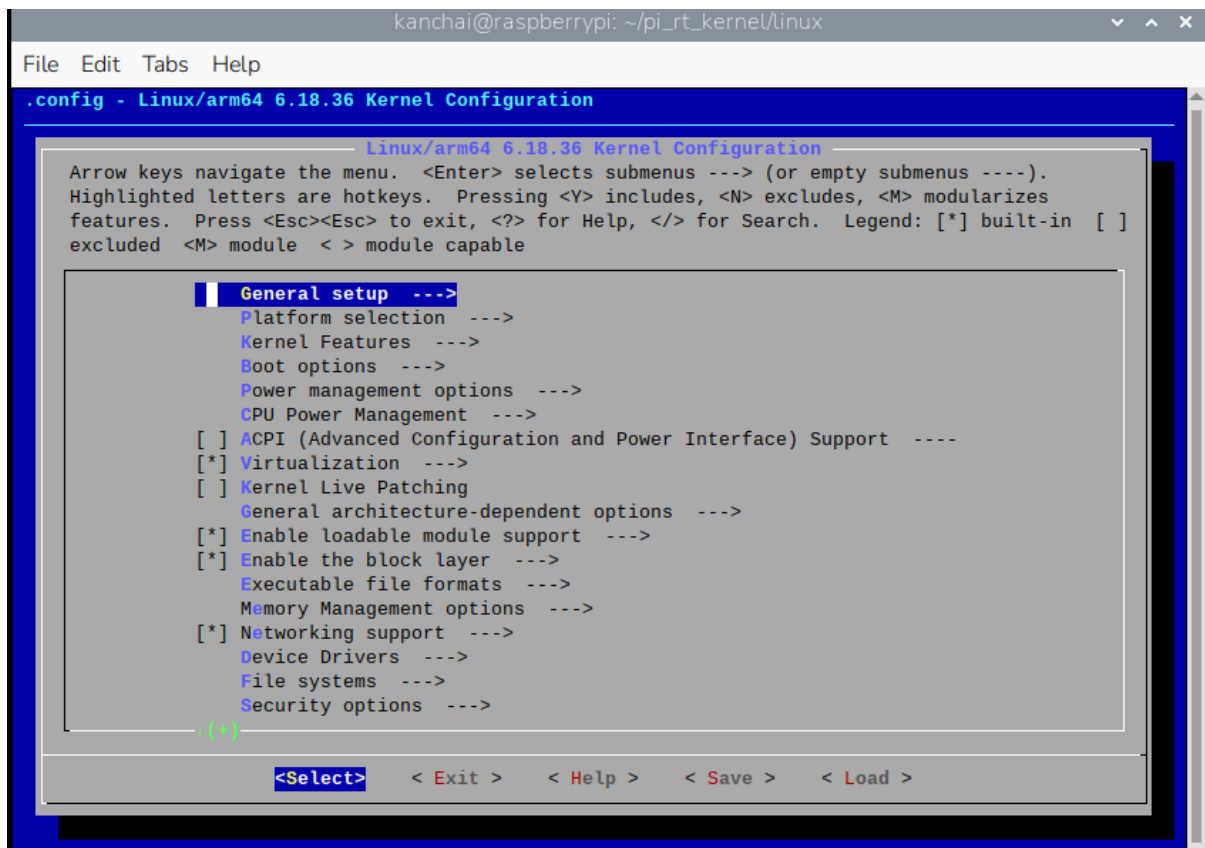
สร้างไฟล์กำหนดค่าสถาปัตยกรรมสำหรับชิป BCM2712 บน Compute Module 5 (CM5):

Bash

```
make bcm2712_defconfig
```

```
make menuconfig
```

ตัวอย่างหน้า UI



เมื่อเข้าสู่หน้าจอเมนูกราฟิกสีน้ำเงิน-เทา (Menuconfig Interface) ดำเนินการตั้งค่าเฉพาะเจาะจงเพื่อเปิดระบบ Real-Time ดังต่อไปนี้:

หัวข้อหลัก (Main Menu), เมนูกลุ่มย่อย (Sub-menu), พารามิเตอร์ที่ต้องแก้ไข (Parameter),

วัตถุประสงค์ (Purpose)

General Setup-> Preemption Model, เปลี่ยนเป็น Fully Preemptible Kernel (Real-Time),

เปิดใช้งานกลไกขัดจังหวะการทำงานในระดับโครงสร้างหลักของ Kernel เพื่อลด Latency โดยการเปิดแบบนี้ [*] Fully Preemptible Kernel (Real-Time)

General Setup, Timers subsystem, เปิดใช้งาน [*] High Resolution Timer Support,

รองรับสัญญาณนาฬิกาที่มีความแม่นยำสูงระดับไมโครวินาทีสำหรับงาน Real-Time

CPU Power Management -> CPU Frequency scaling -> Default CPUFreq

governor เปลี่ยนเป็น (x)performance, บังคับให้ CPU วิ่งที่ความเร็วสูงสุดตลอดเวลาป้องกันความหน่วงจากการปรับความถี่ตามภาระงาน

Kernel hacking -> Compile-time checks and Compiler options -> Debug information

ติ๊ก (x) Disable debug information, ปิดการสร้างไฟล์สัญลักษณ์ตรวจสอบ บั๊ก ช่วยลดขนาดไฟล์ Kernel ลง 10 เท่า และเพิ่มความเร็วในการคอมไพล์

กดเลื่อนไปที่ < Save > ด้านล่าง แล้วกด Enter เพื่อบันทึกไฟล์ .config

กด < Exit > ออกมา แล้วเริ่มทำการคอมไพล์และการติดตั้ง

(Compilation & Installation)

4. กระบวนการคอมไพล์และการติดตั้งระบบ (Compilation & Installation)

เมื่อสิ้นสุดการบันทึกค่าลงในไฟล์ .config

ให้ดำเนินการคอมไพล์ซอร์สโค้ดโดยใช้หน่วยประมวลผลทั้ง 4 คอร์ของ Compute Module 5 (CM5):

Bash

```
make -j4 Image modules dtbs
```

หลังจากกระบวนการคอมไพล์เสร็จสิ้นโดยไม่มีข้อผิดพลาด ให้ทำการติดตั้ง Modules

และย้ายระบบไฟล์อิมเมจไปยังพาร์ทิชันระบบบูตหลัก (Boot Firmware Partition) ของเครื่อง:

Bash

```
sudo make modules_install
```

```
sudo cp arch/arm64/boot/Image /boot/firmware/kernel8-rt.img
```

```
sudo cp arch/arm64/boot/dts/broadcom/*.dtb /boot/firmware/
```

```
sudo cp arch/arm64/boot/dts/overlays/*.dtbo /boot/firmware/overlays/
```

```
sudo cp arch/arm64/boot/dts/overlays/README /boot/firmware/overlays/
```

4.1 การกำหนดสิทธิ์การบูต (Boot Configuration)

ทำการเปิดไฟล์กำหนดค่าการบูตระบบของ Raspberry Pi:

Bash

```
sudo nano /boot/firmware/config.txt
```

เพิ่มคำสั่งเงื่อนไขโครงสร้างไว้ที่ส่วนท้ายสุด เพื่อกำหนดให้บอร์ดเรียกใช้งานอิมเมจ Real-Time

Kernel ตัวใหม่ที่สร้างขึ้นทดแทนระบบเดิม:

[all]

kernel=kernel8-rt.img

```
[cm4]
# Enable host mode on the 2711 built-in XHCI USB controller.
# This line should be removed if the legacy DWC2 controller is required
# (e.g. for USB device mode) or if USB support is not required.
otg_mode=1

[cm5]
dtoverlay=dwc2,dr_mode=host

[pi5]
dtoverlay=nospi10

[all]
kernel=kernel8-rt.img
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Ex
^X Exit ^R Read File ^\ Replace ^U Paste ^J Ju

(ถ้าอยากกลับมาใช้ PREEMPTเดิมให้ใส่ # แบบนี้ #kernel=kernel8-rt.img)

บันทึกไฟล์กด ctrl + o แล้วกด Enter

แล้วกด ctrl + x เพื่อ ออก แล้วทำการreboot* ทำการบันทึกไฟล์และรันคำสั่งรีบูตระบบ: **sudo reboot**

5. การทดสอบและการวัดผลประสิทธิภาพ (Benchmarking)

เมื่อระบบเปิดใช้งานขึ้นมาใหม่ สามารถดำเนินการตรวจสอบสถานะความเป็น Real-Time

ผ่านคำสั่งระบบ:

Bash

uname -a

ผลลัพธ์ที่ถูกต้องควรปรากฏคำว่า PREEMPT_RT ขนาบข้างเวอร์ชันระบบ

cat /sys/kernel/realtime

ผลลัพธ์ที่แสดงต้องส่งกลับค่าเป็นเลข "1" เพื่อยืนยันสถานะการเปิดใช้งานสมบูรณ์